

CSC 4553

Fall 2026 - Day 12

Admin

- Programming assignment 2 (schedfit) due Monday, 18:00
- Lab due Friday, 18:00
- Quiz due Monday, 18:00

Synchronization

Questions to consider

- What are the requirements for a critical section synchronization solution, and how do bounded waiting and progress differ?
- Why do naive implementations like a simple boolean lock fail, and what must happen atomically instead?
- What are the practical limitations of Peterson's solution, and why don't they work well on modern architectures?

Problem

- Two processes accessing a reservation system, where the number of open seats is tracked by a counter
 - A: Releasing Seat
 - B: Reserving Seat
- Counter at 10; Process A `counter++ (10 -> 11);` Process B `counter-- (10 -> 9);`
- What could go wrong?

```
1  mov 0x8049a1c, %eax
2  add $0x1, %eax
3  mov %eax, 0x8049a1c
4
5  mov 0x8049a1c, %eax
6  sub $0x1, %eax
7  mov %eax, 0x8049a1c
```

Race conditions

- If interleaved, whoever goes last wins. Either way, 11 or 9 is an incorrect value. This is a **race condition**
- Very common in multi-programming environments: allocating disk blocks or memory pages, writing to a file, network buffers, etc.
- How can we solve race conditions?
 - **Mutual Exclusion** (one at a time) on **Critical Sections** (the contention data/code)

Challenges we face

- How large should a critical section be?
- How long can a process be in a critical section
 - What happens if they crash while within a critical section?
- How do we implement? Who enforces entrance into critical sections?

Beware naïve implementations

- Why not use a simple bool?

```
1 bool lock = FALSE
2 do {
3     while (lock == TRUE);
4     lock = TRUE;
5     //CRITICAL SECTION
6     lock = FALSE;
7 }while(TRUE);
```

- This implementation requires TWO operations: a read (test) and a write (set) that must be atomic

Critical section solution requirements

- Mutual exclusion
- Progress
- Bounded waiting

Requirement 1: Mutual exclusion

- **Safety** property for the system::At most one process in the CS at any instant
- Traffic light: cross streets are **never** green at the same time. One double green, even for an instant, breaks it
- To show it's violated: find **one** interleaving that puts two processes in the CS
- To show it holds: argue that *no* interleaving can (one lucky run proves nothing)
- The naïve **bool** lock fails: both processes can read **lock == FALSE** before either one sets it

Step	P_0	P_1	lock
1	reads lock : F, leaves while		F
2	<i>preempted</i>	reads lock : F, leaves while	F
3		lock = TRUE , enters CS	T
4	<i>resumes</i> : lock = TRUE (no re- check)	in CS	T
5	enters CS	in CS	T

Both in the CS.

Requirement 2: Progress

- **Liveness** property for the system: if the CS is empty and someone wants in, **someone** gets in
 - Traffic light: if the intersection is empty and a car is waiting, **some** direction eventually gets a green. The all-red light fails this
- Two parts to the rule:
 - A process that **isn't trying to enter** can't keep anyone else out
 - If several processes are trying to enter, one of them gets in. They can't all wait on each other forever
- Test for a violation: Is the CS empty? Is someone trying to get in? Are they stuck anyway?
 - Three yeses means Progress is violated

Progress example: strict alternation

```
1 // shared: int turn = 0;
2 // P_i's code (j is the other)
3 do {
4   while (turn != i);
5   //CRITICAL SECTION
6   turn = j;
7   //REMAINDER
8 } while (TRUE);
```

Step	P_0	P_1	turn
1	in CS	in remainder	0
2	turn = 1	in remainder	1
3	wants in again	in remainder	1
4	stuck in while	in remainder	1

- Mutual exclusion ✓: turn can't be 0 and 1 at once
- CS empty ✓, P_0 wants in ✓, P_0 stuck ✓
- Progress ✗

Requirement 3: Bounded waiting

- **Liveness** property for *each process*: once P_i asks to enter, only a bounded number k of other entries can happen before P_i gets its turn
 - Progress says *someone* gets in. Bounded waiting says *you* eventually get in
 - The bound is a **count of entries**, not a time. We can't assume anything about CPU speed
- Traffic light: a car on the side street gets a green within k light cycles. A light that always favors the busy main road starves the side street, even though traffic is always moving
- Violating it means **starvation**
- Fix idea: take a number (FIFO), like a deli counter. Then $k = n - 1$

Bounded waiting example: P_0 always wins

- Suppose a lock (whatever its implementation) always lets P_0 win a tie
- Mutual exclusion ✓
- Progress ✓: the CS is never idle while someone waits
- Bounded waiting ✗: P_1 can lose every time

Time	P_0	P_1
1	in CS	requests entry
2	exits, re-requests	waiting
3	wins tie , in CS	waiting
4	exits, re-requests	waiting
5	wins tie , in CS	waiting
...	...	waiting forever

Progress vs. bounded waiting

	Progress	Bounded waiting
Protects	the system	each process
Promise	someone gets in	you get in
Traffic light	an empty intersection with a waiting car doesn't stay red	the side street gets a green within k cycles
Failure	CS sits empty while processes wait (deadlock)	CS stays busy , but one process never gets a turn (starvation)

- Quick test: **is the CS being used?**
 - Empty and someone's stuck → Progress violated
 - Busy, but the same process keeps losing → Bounded waiting violated

Discussion: Break this lock

- This is P_i 's code (swap i and j for P_j). Which of the three requirements does it **meet**, and which does it **violate**?
- How would you fix it?

```
1 // shared: bool flag[2] = {FALSE, FALSE};
2 do {
3     flag[i] = TRUE;
4     while (flag[j]);
5     //CRITICAL SECTION
6     flag[i] = FALSE;
7     //REMAINDER
8 } while (TRUE);
```

Peterson's solution

- Two processes: P_i and P_j
- Two shared data items: `int turn; bool flag[2];`

```
1  Pi code:
2  do {
3      flag[i] = TRUE;           // I want to enter critical section
4      turn = j;                 // let Pj go first if it wants
5      while (flag[j] && turn == j); // busy wait
6      //CRITICAL SECTION
7      flag[i] = FALSE;         // Im leaving critical section
8      //REMAINDER
9  }while(TRUE);
```

- `turn` is a tie-breaker: if both want in, the last one to write it lets the other go first
- if `flag[i] == true` → P_i is ready for the critical section
- To enter: P_i sets `flag[i] = true` and `turn = j`
 - i.e., P_i demurs to P_j ; if they are both ready `turn` will get set to?
 - Whoever runs last sets the `turn` to the other process

Peterson's solution: mutual exclusion

```
1  Pi code:
2  do {
3      flag[i] = TRUE;           // I want to enter critical section
4      turn = j;                 // let Pj go first if it wants
5      while (flag[j] && turn == j); // busy wait
6      //CRITICAL SECTION
7      flag[i] = FALSE;         // Im leaving critical section
8      //REMAINDER
9  }while(TRUE);
```

- Can both processes be in the CS at the same time?
 - P_i gets past its **while** only if **flag[j] == FALSE** or **turn == i**
 - Suppose both are in the CS. Look at who wrote **turn last**, say P_i (it wrote **turn = j**)
 - P_j already set **flag[j] = TRUE** before its own **turn = i**, and nobody touches **turn** again
 - So P_i sees **flag[j] == TRUE && turn == j** and spins until P_j leaves. Contradiction
 - Both say “after you”; the **last** one to say it is the one who waits

Peterson's solution: progress

```
1  Pi code:
2  do {
3      flag[i] = TRUE;           // I want to enter critical section
4      turn = j;                 // let Pj go first if it wants
5      while (flag[j] && turn == j); // busy wait
6      //CRITICAL SECTION
7      flag[i] = FALSE;          // Im leaving critical section
8      //REMAINDER
9  }while(TRUE);
```

- If the CS is empty and someone wants in, does someone get in?
 - P_j not trying: `flag[j]` is FALSE, so P_i walks straight in (strict alternation failed this)
 - Both trying: `turn` is either `i` or `j`, so one of the two `while` conditions is false and that process enters (flags-only deadlocked here)

Peterson's solution: bounded waiting

```
1  Pi code:
2  do {
3      flag[i] = TRUE;           // I want to enter critical section
4      turn = j;                 // let Pj go first if it wants
5      while (flag[j] && turn == j); // busy wait
6      //CRITICAL SECTION
7      flag[i] = FALSE;          // Im leaving critical section
8      //REMAINDER
9  }while(TRUE);
```

- Once P_i is waiting, how many times can P_j get in first?
 - At most **once**. To get back in, P_j must set **turn = i**, which lets P_i through and makes P_j wait

Peterson's solution (cont'd)

```
1 is code:
2 do {
3   flag[i] = TRUE;           // I want to enter critical section
4   turn = j;                 // let Pj go first if it wants
5   while (flag[j] && turn == j); // busy wait
6   //CRITICAL SECTION
7   flag[i] = FALSE;          // Im leaving critical section
8   //REMAINDER
9 }while(TRUE);
```

- Do you see any issues with this solution?
 - Limitation: Only works for two processes
 - Modern architectures may also break this solution
 - Processors and/or compilers can reorder reads and writes that have no dependencies
 - Multithreaded applications could lead to issues

What isn't clear?

Comments? Thoughts?